Ruby Idioms

A number of individual Ruby features can be combined in interesting ways. We use these common Ruby idioms in this book:

Methods such as `empty!` and `empty?`
: Ruby method names can end with an exclamation mark (a *bang method*) or a question mark (a *predicate method*). Bang methods normally do something destructive to the receiver. Predicate methods return `true` or `false`, depending on some condition.

`a || b`
: The expression `a || b` evaluates `a`. If it isn't `false` or `nil`, then evaluation stops and the expression returns `a`. Otherwise, the statement returns `b`. This is a common way of returning a default value if the first value hasn't been set.

`a ||= b`
: The assignment statement supports a set of shortcuts: `a op= b` is the same as `a = a op b`. This works for most operators:

```
count += 1          # same as count = count + 1
price *= discount   #         price = price * discount
count ||= 0         #         count = count || 0
```

So, `count ||= 0` gives `count` the value `0` if `count` is nil or false.

`obj = self.new`
: Sometimes a class method needs to create an instance of that class:

```
class Person < ApplicationRecord
  def self.for_dave
    Person.new(name: 'Dave')
  end
end
```

This works fine, returning a new `Person` object. But later, someone might subclass our class:

```
class Employee < Person
```